

FACULDADE DE ENGENHARIA DA UNIVERSIDADE DO PORTO

<Dissertation Title>

<Fernando Azeredo>

WORKING VERSION



Mestrado em Engenharia Informática e Computação

Supervisor: Prof. <António Sá Pinto>

June 17, 2026

Resumo

Este documento ilustra o formato a usar em dissertações na Faculdade de Engenharia da Universidade do Porto (FEUP). São dados exemplos de margens, cabeçalhos, títulos, paginação, estilos de índices, etc. São ainda dados exemplos de formatação de citações, figuras e tabelas, equações, referências cruzadas, lista de referências e índices. Este documento não pretende exemplificar conteúdos a usar. É usado o *Loren Ipsum* para preencher a dissertação. Lorem ipsum dolor sit amet, FEUP consectetur adipiscing elit. Etiam vitae quam sed mauris auctor porttitor. Mauris porta sem vitae arcu sagittis facilisis. Proin sodales risus sit amet arcu. Quisque eu pede eu elit pulvinar porttitor. Maecenas dignissim tincidunt dui. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Donec non augue sit amet nulla gravida rutrum. Vestibulum ante ipsum primis in faucibus orci luctus et ultrices posuere cubilia Curae; Nunc at nunc. Etiam egestas. Donec malesuada pede eget nunc. Fusce porttitor felis eget mi mattis vestibulum FEUP. Pellentesque faucibus. Cras adipiscing dolor quis mi. Quisque sagittis, justo sed dapibus pharetra, lectus velit tincidunt eros, ac fermentum nulla velit vel sapien. Vestibulum sem mauris, hendrerit non, feugiat ac, varius ornare, lectus. Praesent urna tellus, euismod in, hendrerit sit amet, pretium vitae, nisi. Proin nisl sem, ultrices eget, faucibus a, feugiat non, purus. Etiam mi tortor, convallis quis, pharetra ut, consectetur eu, orci. Vivamus aliquet. Aenean mollis fringilla erat. Vivamus mollis, purus at pellentesque faucibus, sapien lorem eleifend quam, mollis luctus mi purus in dui. Maecenas volutpat mauris eu lectus. Morbi vel risus et dolor bibendum malesuada. Donec feugiat tristique erat. Nam porta auctor mi. Nulla purus. Nam aliquam.

Palavras-chave: keyword1 • keyword2 • keyword3 • keyword4

Abstract

Here goes the abstract written in English. Lorem ipsum dolor sit amet, consectetur adipiscing elit. Sed vehicula lorem commodo dui Faculdade de Engenharia da Universidade do Porto (FEUP). Fusce mollis feugiat elit. Cum sociis natoque penatibus et magnis dis parturient montes, nascetur ridiculus mus. Donec eu quam. Aenean consectetur odio quis nisi. Fusce molestie metus sed neque. Praesent nulla. Donec quis urna. Pellentesque hendrerit vulputate nunc. Donec id eros et leo ullamcorper placerat. Curabitur aliquam tellus et diam. Ut tortor. Morbi eget elit. Maecenas nec risus. Sed ultricies. Sed scelerisque libero faucibus sem FEUP. Nullam molestie leo quis tellus. Donec ipsum. Nulla lobortis purus pharetra turpis. Nulla laoreet, arcu nec hendrerit vulputate, tortor elit eleifend turpis, et aliquam leo metus in dolor. Praesent sed nulla. Mauris ac augue. Cras ac orci. Etiam sed urna eget nulla sodales venenatis. Donec faucibus ante eget dui. Nam magna. Suspendisse sollicitudin est et mi. Fusce sed ipsum vel velit imperdiet dictum. Sed nisi purus, dapibus ut, iaculis ac, placerat id, purus. Integer aliquet elementum libero. Phasellus facilisis leo eget elit. Nullam nisi magna, ornare at, aliquet et, porta id, odio. Sed volutpat tellus consectetur ligula. Phasellus turpis augue, malesuada et, placerat fringilla, ornare nec, eros. Class aptent taciti sociosqu ad litora torquent per conubia nostra, per inceptos himenaeos. Vivamus ornare quam nec sem mattis vulputate. Nullam porta, diam nec porta mollis, orci leo condimentum sapien, quis venenatis mi dolor a metus. Nullam mollis. Aenean metus massa, pellentesque sit amet, sagittis eget, tincidunt in, arcu. Vestibulum porta laoreet tortor. Nullam mollis elit nec justo. In nulla ligula, pellentesque sit amet, consequat sed, faucibus id, velit. Fusce purus. Quisque sagittis urna at quam. Ut eu lacus. Maecenas tortor nibh, ultricies nec, vestibulum varius, egestas id, sapien. Donec hendrerit. Vivamus suscipit egestas nibh. In ornare leo ut massa. Donec nisi nisl, dignissim quis, faucibus a, bibendum ac, diam. Nam adipiscing hendrerit mi. Morbi ac nulla. Nullam id est ac nisi consectetur commodo. Pellentesque aliquam massa sit amet tellus. Vivamus sodales aliquam leo.

Keywords: keyword1 • keyword2 • keyword3 • keyword4

Acknowledgements

Aliquam id dui. Nulla facilisi. Nullam ligula nunc, viverra a, iaculis at, faucibus quis, sapien. Cum sociis natoque penatibus et magnis dis parturient montes, nascetur ridiculus mus. Curabitur magna ligula, ornare luctus, aliquam non, aliquet at, tortor. Donec iaculis nulla sed eros. Sed felis. Nam lobortis libero. Pellentesque odio. Suspendisse potenti. Morbi imperdiet rhoncus magna. Morbi vestibulum interdum turpis. Pellentesque varius. Morbi nulla urna, euismod in, molestie ac, placerat in, orci.

Ut convallis. Suspendisse luctus pharetra sem. Sed sit amet mi in diam luctus suscipit. Nulla facilisi. Integer commodo, turpis et semper auctor, nisl ligula vestibulum erat, sed tempor lacus nibh at turpis. Quisque vestibulum pulvinar justo. Class aptent taciti sociosqu ad litora torquent per conubia nostra, per inceptos himenaeos. Nam sed tellus vel tortor hendrerit pulvinar.

Fusce gravida placerat sem. Aenean ipsum diam, pharetra vitae, ornare et, semper sit amet, nibh. Nam id tellus. Etiam ultrices.

<O Nome do Autor>

Contents

1	Introduction	1
1.1	Contextualization	1
1.2	Developed Project	1
1.3	Motivation	1
1.4	Thesis Structure	1
1.5	Thesis Structure	1
2	Literature Review	2
2.1	Introduction	2
2.2	Visualization Types	2
2.2.1	Sheet Music and Score Images	2
2.2.2	Symbolic Representations	3
2.2.3	Audio Representations	4
2.3	MIR Visualization Tools	11
2.4	Audio-to-Score Alignment	12
2.5	Summary	14
3	Developed Solution	15
3.1	Developed Iterations	15
3.1.1	BeatMapJS	16
3.1.2	MEI-Basic-Edit	16
3.1.3	BeatPrecision, a PP fork	17
3.1.4	Score-Alignment.py	17
3.1.5	Score-Alignment.js	17
3.2	MIRVisualScore	18
4	Implementation	19
4.1	Dependencies and Setup	19
4.2	Required Files	20
4.2.1	Getting the Warped Score	20
4.2.2	Getting the MAPS file	20
4.2.3	Providing BT data	21
4.2.4	Summarizing Required Files	22
4.3	Calling Modules, Methods and Functions	22
4.4	Creating the Audio Visualizations	25
4.5	Score and Data Alignment	25
4.6	Combining Visualizations	25
4.7	Conclusion	25

5	Conclusions and Future Work	26
5.1	Achievements	26
5.2	Future Work	26
	References	27
A	Lorem Ipsum	28
A.1	What is <i>Lorem Ipsum</i> ?	28
A.2	Where does <i>Lorem</i> come from?	28
A.3	Why is <i>Lorem</i> used?	29
A.4	Where can you find examples?	29

List of Figures

2.1	Example of an audio waveform.	5
2.2	Example of audio Spectrum.	6
2.3	Example of Linear spectrogram.	7
2.4	Example of Logarithmic spectrogram.	8
2.5	Example of Mel spectrogram.	9
2.6	Example of a CQT.	10
2.7	Example a Chromagram.	11

List of Tables

List of Acronyms

FEUP Faculdade de Engenharia da Universidade do Porto

Chapter 1

Introduction

1.1 Contextualization

Music Information Retrieval (MIR) is a research field that develops computational methods to analyze, interpret, and retrieve musical data from performance recordings. MIR is heavily reliant on data, and thus, visualization plays a crucial role; how and what we visualize in data can shape not only the conclusions one makes, as well as providing guidance on how to further progress. The problem of visualization in MIR is especially critical because music is inherently subjective and abstract. Music is also a uniquely auditory medium, translating it into visual form is a challenge that spans millennia, with many attempts at "musical written" languages throughout history. Yet this task remains unfolding. For MIR researchers, this means working with diverse data representations, each capturing different aspects of music.

1.2 Developed Project

For this thesis we developed MIRVisualScore, a tool that combines multiple state-of-the-art and well established MIR resources into one single Python-based object-oriented (OO) tool. MIRVisualScore takes advantage of ScoreWarp, librosa, matplotlib, and BeatThis, in order to produce score aligned audio visualizations with overlaid algorithm outputs. This tool is meant

1.3 Motivation

1.4 Thesis Structure

1.5 Thesis Structure

Chapter 2

Literature Review

2.1 Introduction

Music Information Retrieval (MIR) is the field of research that focuses on the development of algorithms and systems to analyze, retrieve, and organize music data. MIR bridges a wide range of disciplines, including: musicology, signal processing, machine learning, and human-computer interaction. [5]

2.2 Visualization Types

We can sum up all music related visualizations into three categories:

1. Sheet Music and Score Images
2. Symbolic Representations
3. Audio Representations

2.2.1 Sheet Music and Score Images

Sheet music describes musical work using a formal language based on musical symbols and letters, thus, this kind of music representations always require a level of music literacy from the performing musician. Furthermore sheet music usually does not induce explicit information on how a given piece should be performed, it is intended for the musician to have a certain level of interpretation liberty. This can include the variance of tempo, articulation and dynamics among others.

Sheet music also comes in a variety of forms that vary throughout History and cultures. For our purposes we are going to focus on the western standard of music notation. Still, within western music, there can be vastly different forms of notation, however, most are based on a staff which sets five horizontal lines and four spaces, each representing a different musical pitch. Musical relevant symbols are put inside the staff to denote pitch, temporal and expressive indications. [6]

2.2.2 Symbolic Representations

Symbolic Representations are machine-readable formats where musical events and expressions are explicitly encoded. These come in various formats, such as MIDI and MusicXML.

Visually we can identify two main applications for this category, these include: piano-roll representations and digital score representations.

Historically piano-roll representations come from so-called “player pianos”, these were self playing pianos that became quite popular in the late 19th to early 20th century. This was achieved by a mechanism that would hit keys of the piano through a continuous roll of paper with perforations. The roll would move over a reading system (known as a tracker bar) that would trigger the musical actions accordingly.

This concept of a continuous and explicit representation of music performance would later prove useful for digital applications. Such is the case for the Piano-Roll.

The **Piano-Roll** is a staple on any DAW as a visual and interactive tool to write and edit musical events in the form of MIDI notes and attributes (such as velocity and other expressive controls).

MusicXML refers to the XML-based language to transcribe score music into the digital domain. There are multiple sub-variants of the MusicXML format for specific applications, however all share an XML based language underneath.

MusicXML was developed to serve as a universal format for storing and sharing music scores across different music notation applications. Music notation digital editors then employ proprietary variants of this format for addressing the program’s specific characteristics. In some cases these may not even be xml based, but MusicXML format can be considered the universal format. These include, among others:

- **.mscz and .mscx**: These are MuseScore specific formats named for "MuseScore Format" and "Uncompressed MuseScore" format respectively. Both xml based.
- **.dora** - "Dorico" specific format (non xml based)
- **.sib** - "Sibelius" proprietary format of company Avid (also non xml based)
- **.mei** - The "Music Encoding Initiative" (MEI). XML based.

The .mei format is the one that is essential for this thesis and the most useful for MIR purposes. MEI stands for both the XML-based score format in itself (.mei), and the community that develops it. Citing from the MEI website’s introduction:

“The Music Encoding Initiative (MEI) is a community-driven, open-source effort to define a system for encoding musical documents in a machine-readable structure. MEI brings together specialists from various music research communities, (...) to define best practices for representing a broad range of musical documents and structures.”[1]

Important to note that MEI focuses on a "machine-readable structure". This structure relies on unique identifiers (@xml:id) that make every element in the score individually addressable,

from individual noteheads to complex spanning elements like slurs and ties. However, more than a music score format, MEI also serves to keep various notation, performance and interpretation data not only stored but also linked to individual score elements. This is why the MEI format and its community are so essential to MIR as a whole.

That being said, MEI is a purely data driven project, and thus in order for it to be visualized or interacted with it needs to be decoded into a "human-readable format". That's where Verovio comes in. The MEI and Verovio projects are closely inter-linked for that reason. While MEI defines how musical information is stored, Verovio is the primary tool used to transform that data into interactive visualizations and performance-ready outputs.

Verovio is a comprehensive open-source library designed to serve as a native typesetting engine for the MEI format. Verovio's primary role is to engrave MEI data directly into SVG without losing rich metadata through intermediate format conversions. The choice for making it into an SVG is an essential one for multiple reasons, quoting from Verovio's reference book:

"In a web environment, the addressability can be used for highlighting graphical elements such as notes or any other music symbols. One additional characteristic of SVG is that its XML tree can be structured as desired, and an innovative design feature of Verovio was to go further in the structuring of the output by leveraging this characteristic. Since Verovio implements the MEI structure internally, this key feature of SVG made it possible to preserve the MEI structure in the design of the SVG output in Verovio. Preserving the MEI structure in the SVG output is a considerable overhead in the rendering process but makes it a unique feature of Verovio.

"As a result, Verovio output in SVG is not the end of a unidirectional rendering process. Quite on the contrary, it should instead be seen as an intermediate layer standing between the MEI encoding and its rendering that can act as the cornerstone for a bi-directional interaction: from the encoding to the notation, but also from the notation to the encoding through the user interface." *repertoireinternationaldessourcesmusicalesVerovioReferenceBook2021*

To further demonstrate the usefulness of the MEI and Verovio, I'll present next a quote from the article relative to Piano Precision (PP) a GUI software for audio-to-score visualization, we will explore this tool further in the next section.

"Technically, what makes MEI especially useful is that it can be rendered to graphical SVG output while retaining its semantically meaningful XML hierarchy and unique identifiers for every score element (e.g., ties, notes). Piano Precision renders the scores into SVG output using Verovio, an open-source toolkit for visualizing MEI data. This makes it easy to locate interesting objects in a score and, e.g., highlight them in the rendering. The result is interactive images in which users can flip pages, zoom in and out of a score, and click on elements in a score to jump to their playback positions." *jiangPIANOPRECISIONADVANCING2025*

2.2.3 Audio Representations

Note: The figures in this chapter will follow the same music excerpt from Chopin Op10 n3. They were generated using the AudioVisualizers.py script, which is available in the appendix.

Audio representations are graphs and/or plots that intend to represent sound. These may not be for strictly representing music, the intent is to translate sound from the audible to the visual domain. Audio is a term that refers to an acoustic sound that is turned into an electrical signal. Most audio representations, are thus digitally generated through applying mathematical equations to the audio signal, such as the Fourier Transform.

We can infer at least 3 components that define a given audio. These are, time, frequency and amplitude/energy. An audio visualization will present explicit information of at least 2 of these.

Waveforms are the most common and one of the simplest ways to represent audio. It consists of a single line on a graph in a 2D graph where amplitude is the y-axis and time is in the x-axis. This representation does not provide frequency information, it only shows how amplitude of the audio changes through it's time.



Figure 2.1: Example of an audio waveform.

A spectral visualization, or a spectrum, refers to a graph with only frequency and amplitude/energy representation in a given time window.



Figure 2.2: Example of audio Spectrum.

For displaying the three sound components the plot needs to represent three distinct dimensions. The spectrogram achieves this by having on the y axis the frequency, time on the x-axis and color as substitute for the third axis (amplitude), and in this way displaying all three parameters in a single 2D visual plot. There are multiple weights and variances of the spectrogram that have different applications.

The way frequencies are displayed in a given spectrogram can help understanding audio better for certain situations. There can be a wide variety of spectrogram representations, that can vary in how it's contents are displayed.

The linear spectrogram is the most basic form of spectrogram, where frequencies are displayed in a linear scale. Meaning, frequencies are equally spaced on the y-axis. For music applications, this kind of representation tends to be less useful, as it does not reflect the way humans perceive sound.



Figure 2.3: Example of Linear spectrogram.

The logarithmic spectrogram displays frequencies in a logarithmic scale. This mimics better the way humans perceive sound.



Figure 2.4: Example of Logarithmic spectrogram.

The Mel spectrogram also displays frequencies taking into account the human hearing curve. The difference being, that log spectrogram simply organizes frequencies logarithmically. Whilst the Mel spectrogram organizes frequencies according to the Mel scale, which is a perceptual scale of pitches judged by listeners to be equal in distance from one another. For music related applications this type of spectrogram is much more useful, since we can more clearly distinguish in between musical pitches.



Figure 2.5: Example of Mel spectrogram.

Next I'll introduce the Constant-Q Transform (CQT) and the Chromagram. These are, to some extent, visual representations that derive from the spectrogram, the difference being that whilst the later shows frequency data, the CQT and chromagram can be used to visualize note data. To be exact, tone height in the case of the CQT and Chroma for the Chromagram.

“(...) a pitch can be separated into two components, which are referred to as tone height and chroma. The tone height refers to the octave number and the chroma to the respective pitch spelling attribute contained in the set {C, C $\sharp$, D,(...), B}.” [5]

The CQT organizes its frequency content using geometrically spaced frequency bins. This is made in order to resemble the equal-tempered system. More commonly, the CQT can be interpreted as a Piano-Roll like representation.[p. 3, 8]



Figure 2.6: Example of a CQT.

While the CQT can distill the frequency spectrum into bins that correlate to individual notes, the chromagram takes this a step further by visually summing the energy of each note across all octaves. Thus in the end we get a plot divided into 11 lines, each representing a give note. In it's essence this type of visualization serves to see the usage of the same notes across the recording. A chromagram can be useful for determining harmonic and scale related data.



Figure 2.7: Example a Chromagram.

[3–5, 8]

2.3 MIR Visualization Tools

Within MIR we can identify two main ways for visualization of audio for MIR tasks, these are either by using GUI based tools, or by using programming-based workflows.

For GUI based tools two stand-out, these being Audacity and especially Sonic Visualizer (SV). Audacity is one of the most popular audio editors; it is designed for general audio related tasks from audio editing, recording, processing, and visualization. On the other hand, SV is a strictly designed program for audio visualization and analysis. Whilst Audacity the most "user friendly" and popular program, SV is meant for scientific and specialized audio visualization tasks.

Important to note that both software solutions are free and open-source based. These are important features, since they allow for users to easily access them, but also for researchers and experts to modify them and thus, contribute to the development of SV and new SV-like tools.

One such example is Piano Precision, a SV based GUI tool for visualizing score-aligned spectrograms.

“The Piano Precision UI is adapted from Sonic Visualizer, with an additional area displaying a score (...) The interface is designed to help streamline the processes of loading a digital score and a performance recording, editing and visualizing annotations (...), and exporting score-aware annotations.”[2]

While modern GUI-based visualization tools may be powerful, they might not provide features that are required for state-of-the-art research and development. MIR research often requires highly specialized visualizations tailored to specific research questions that pre-built tools simply do not support. Understandably, the development of these programs simply cannot keep up with the constantly evolving advancements in the field.

That is why it is also quite common for researchers to develop programming-based methods for creating specialized visualizations. Researchers might need to integrate multiple data types (audio, scores, annotations, computational outputs) in ways that GUI tools cannot replicate.

Moreover, code-based workflows can be versioned, shared, and included in publications, ensuring reproducibility this is an essential requirement for academic research.

Another argument is that MIR is increasingly geared towards machine learning solutions and algorithms. The need to work with large music datasets and integrate machine learning applications for training, feature extraction, and other related tasks, explains the reliance on code-based visualization workflows.

Furthermore, the fact that MIR is fundamentally a computational research field more broadly justifies this approach.

Currently, Python is the go-to language for MIR visualization tasks. Key libraries such as Librosa, Matplotlib, and Madmom, along with APIs such as Modusa and tools like Jupyter Notebooks, are common in MIR research.

2.4 Audio-to-Score Alignment

The task of Audio-to-Score Alignment (score matching or score alignment) consists of “synchronizing an audio recording of a musical piece with the corresponding symbolic score” carabias-ortiAUDIOSCOREALIGNMENTa. There are multiple applications for such a feature, from automatic score following for music performance or for guided listening, to interactive navigation between audio and score (useful for ex. audio editors) to MIR research and development.

There are two modes of alignment: offline alignment and online alignment. Offline alignment, consists of accessing a musical recording and performing the alignment afterwards, this can be useful for tasks such as musical analysis and audio editing. Online alignment (also known as also known as score following), processes the data in real-time while the audio signal is being received, this is useful for tasks like autonomous accompaniment of the score while a performance is occurring. Offline alignment is able to use the full length of the recording in order to determining the optimal alignment, this is because it can “look into the future” in order to establish the optimal matching. Because of this, Online alignment can’t provide the same level of detail unless

some deliberate latency is introduced into the system thus, online modes need to take into account precision trade-offs.

Besides online and offline modes of score matching, the alignment in itself can be done to various levels of the performance or score these include: Note-Level Beat-level Segment

Note-level alignments are score matching processes that aim to synchronize at the individual note level. At its core, note-level alignments are systems based on note and onset/offset detection and linking. In MIR literature it can be also referred as “fine-grained alignment” due to the level of precision that these methods require. Contemporary Audio-to-Score MIR research focuses on this type of alignment not only because of the precision element but also because it can retrieve data that is relevant for other MIR fields and tasks. However, pure note-level alignments might falter if the performance deviates from the score i.e. if the performer skips, repeats or adds notes and sections of the score. That’s why some score matching solutions try to employ, on top of fine-grained alignments, other methods.

Piano Precision (PP) software provides a note-level score aligned visualization. While a recording is playing PP shows the current playback position via highlighting the element being played in the score note by note. This enables annotations to be mapped to both the audio and individual score elements simultaneously. The authors note the usefulness of this tool for performance researchers. jiangPIANOPRECISIONADVANCING2025

Beat-Level alignments aim to follow along beat and/or measures dictated in the score, and then linking them to the performance. Early score aligners (ex.duanSoundprismOnlineSystem2011) would rely on such methods. Today, score alignment solutions, although mostly are note-level types, still combine Beat-Level alignment layers within their algorithms. A good example of this comes from [7], in it the authors improved on other alignment models, one strategy employed was adding analysis of “alignments at both the beat and the bar level”. The beat level alignment would serve as “anchor points” in order to prevent “cascading errors, as misalignment will not propagate beyond the next anchor point”, they noted that this improved results of automatic alignments.

Segment type alignments, map broad portions of performance audio to corresponding visual blocks in the score. This type of score matching is mostly made manually, it is common to see it in follow-along videos to help viewers listen to the music while visualizing the score. This type of alignment is considered “weak but musically meaningful” jungUnifiedCrossmodalTranslation2026a since segments can be deliberately chosen i.e. to better show melodic or harmonic structure.

ScoreWarp is a relevant visual score alignment tool for this thesis. It presents a score that is tied to a physical time axis where, each element of the score is “physically shifted so that their location linearly coincides with the physical time of their sounding” goeblLetsScoreWarpAgain2025a. ScoreWarp takes advantage of the MEI format and Verovio in order to automatically shift the position of musical elements, while maintaining the semantic integrity of the original score. In the next chapter we are going to explore this tool more in depth, since it became one of the key tools for the development of the project for this thesis.

2.5 Summary

This chapter provided a comprehensive overview of Music Information Retrieval (MIR) visualization techniques and tools, establishing the foundation for understanding audio-to-score alignment.

We began by categorizing music visualizations into three main types: traditional sheet music representations, symbolic machine-readable formats, and audio-based representations. With this we established the importance of the MEI and Verovio as well as the usage of the SVG visual format in this context.

We then examined the contemporary landscape of MIR visualization tools, distinguishing between GUI-based solutions such as Sonic Visualizer and Piano Precision, which provide user-friendly interfaces for audio analysis, and code-based workflows that afford researchers greater flexibility to develop specialized visualizations tailored to specific research questions. Python-based approaches using libraries like Librosa and Matplotlib have become the de facto standard for academic MIR work, particularly due to their integration with machine learning pipelines and reproducibility requirements.

Finally, we introduced the core task of this thesis: Audio-to-Score Alignment. This process synchronizes musical performances with their corresponding symbolic scores. ScoreWarp emerged as a particularly relevant tool, demonstrating how MEI and Verovio can be leveraged to create visually warped scores that maintain semantic integrity while aligning with performance time.

These foundations establish the conceptual and technical backdrop for the practical work presented in subsequent chapters.

Chapter 3

Developed Solution

3.1 Developed Iterations

This thesis started with the proposed title “Interactive Musical Beat Annotation GUI”. The initial thesis proposition was to evaluate modern GUI software solutions used for beat annotation (BA) (such as SV), and build upon them with a focus in beat annotation practices. The proposed goals included:

1. Either develop a web-based annotation system, or advancing the existing SV framework. Emphasizing collaborative features, real-time feedback and build on interaction mechanisms for beat annotation.
2. Integration of ML self-supervised and human-in-the-loop features that provide intelligent annotation suggestions and improve user experience. System inspired on [yamamotoHumanintheLoopAdaptationInteractive2021].
3. Validation through user studies.
4. Publish the developed work as an Open Source Python library/framework. The purpose being, to contribute to MIR research community for further development and integration with other existing frameworks.

The motivation from this proposal came from mostly two identified flaws in this topic: The first is that, although the performance of beat tracking algorithms has been improving significantly in recent years, they mostly rely on deep learning based approaches. The issue with this, is that in order for these systems to be trained they need increasingly large and more complex datasets that, although not always directly (peterAutomaticNoteLevelScoretoPerformance2023), need to some extent human made and/or verified beat annotations. The second one is related to human made beat annotation methods, that for the most part, still is a extensive and time consuming task. Besides that, although SV is a powerful tool, it presents a clear gap for beat annotation, where the integration of newer SOTA BT algorithms is difficult and there have been little to no new added features that improve BA task significantly.[9]

Summarizing, the thesis came from a focus in BT, BA and interactive GUIs to, in the end, focusing on a purely MIR visualization problem. Next we will present different iterations the development phase undertook. This should contextualize the change in scope and motivation of the thesis.

3.1.1 BeatMapJS

Developed from October 2025 until February 2026, BeatMapJS was our first attempt to create a GUI application for beat annotation. Our goal was to experiment and explore interactive BA features, libraries and frameworks while focusing on a web-based solution. Using JS/HTML, BeatThis, and Wavesurfer.JS, we successfully built an interactive spectrogram with a web-based interface that integrated the BeatThis algorithm. The application included beat annotation basic tools, region annotation capabilities, a table for editing tempo annotations and descriptions, and the ability to import and export beat annotations to .txt files (compatible with SV).

Although the program was functional, it was ultimately abandoned. The main reason was that it provided no significant innovative BA features beyond BeatThis integration, and critically, no Python-based library was produced as the backbone of the project, which was an important goal for ensuring the work could be reused by the community.

3.1.2 MEI-Basic-Edit

Running from February until March 2026, this project aimed to create a GUI ecosystem where users could manually adjust score elements while simultaneously viewing an audio visualization layer. We had identified an innovative BA feature, Audio-To-Score Alignment, and hypothesized it would greatly benefit both beat annotation and evaluation tasks. The project served as a learning opportunity to explore MEI and Verovio frameworks along with ScoreWarp.

Implemented in JS/HTML using Verovio, MEI score format, and SVG, the application allowed users to upload a recording and its corresponding MEI score file. A static spectrogram would be generated with the score displayed horizontally below it. Users could interact with individual score elements by clicking and dragging them horizontally to manually align the score with the audio-visual representation. The system worked by rendering both the score and spectrogram into an SVG, allowing direct interaction with the SVG file. The GUI also included an integrated XML text editor for direct manipulation of the SVG score code.

This experiment revealed significant complexity in creating interactive scores—both from UI/UX and technical perspectives. However, it provided invaluable insights into how Verovio and MEI function, as well as SVG structure, knowledge that would prove essential for the developed end solution. The project was ultimately abandoned as it did not produce a reusable Python library and the interaction design was functional enough to justify further development in this direction.

3.1.3 BeatPrecision, a PP fork

In March 2025, we explored building on the PP and SV frameworks to create a BA GUI tool. The rationale was straightforward: PP already provided an embedded score alignment feature and all SV functionality, so adding a few BA-focused improvements seemed like a viable path to developing a relevant annotation tool. Using Qt/C++, we integrated BeatThis and implemented automatic score follow-along in a horizontal manner to synchronize with SV’s spectrogram and audio-visualizations.

While functional, the project revealed significant challenges. The SV/PP codebase proved complex and difficult to manage. Long compile times and the sheer size of the codebase made each iteration increasingly challenging to implement, manage and debug. More importantly, this iteration taught us a critical lesson: we needed to distinguish between the interaction and visualization problems in BA. We realized we could not tackle both simultaneously and would need to choose a direction. Recognizing a clear gap in the field, we opted to focus on the visualization aspect, as we saw greater potential for the thesis to benefit the community in this area.

3.1.4 Score-Alignment.py

By the end of March 2025, we shifted our focus to creating a Python-based visualization tool with score-aligned spectrograms. Our motivation was to concentrate on score alignment in a Python based environment, while exploring the Modusa framework’s capabilities. Using Python, Modusa, and matplotlib, we built an interactive spectrogram.

However, this project was abandoned relatively quickly. Implementing a truly interactive and fluid spectrogram (with zoom, pan, timeline, and playback cursor) proved significantly more challenging than anticipated. While we managed to develop a functional interactive spectrogram, its implementation was clunky and unsatisfactory, leading us to discontinue the project in favor of other approaches.

3.1.5 Score-Alignment.js

In April 2026, we attempted to recreate the last iteration, this time implementing it in JS/HTML. Building on the experience from MEI-Basic-Edit and the initial BeatMapJS project, we aimed to create a comprehensive audio-to-score alignment with spectrogram visualization. Leveraging JS/HTML, Verovio/MEI, and Wavesurfer.js, we developed a system that displayed MEI scores horizontally alongside spectrograms. Timestamps—intended to represent beat annotations—could be directly stored in the MEI file using the `<when>` function, and notes in the score would highlight when hovered over, selected, or associated with a timestamp.

While not fully functional, the application showed promise. However, we made a deliberate decision to shift direction once again. We recognized the need to develop a strictly visualization tool purely in Python with an emphasis on an Object-Oriented approach that could be reused and extended by the community. This pivotal decision marked the beginning of MIRVisualScore development.

3.2 MIRVisualScore

It was only after April 2026 that MIRVisualScore started being actually developed. MIRVisualScore is intended to be a Python OO tool for visualizing recorded performances synchronized with the corresponding music score in a time-aligned manner. This tool allows for custom made visualizations of audio recordings where visual elements are treated as independent layers that can be copied, moved and combined in order to develop custom virtualizations.

The project bases itself in librosa and matplotlib to generate audio-visualizations and data plots. The score used for the symbolic representation layer is rendered through ScoreWarp. The end result is an SVG file that overlays the audio-visualization, data layers and score.

Since this thesis started with a focus on beat annotation, we developed the tool with a beat visualization use case in mind. However, we think that the developed program could be useful to be integrated for other MIR purposes, beat annotation and visualization were just our starting points. By itself the codebase also does not have any user interaction provided, we focused on the visualization aspect, yet the way it was programed, integration of into a GUI or even a web-based solution is viable.

The codebase integrates BeatThis the as-of-now SOTA BT algorithm. Through it's implementation we demonstrate how a given BT model can be integrated to our system in order to produce beat data visualizations.

Our main goal is that, this codebase may be easily integrated into other relevant projects to generate visualizations. Hence why we developed it as a Python Based OO tool that is based on commonly used MIR libraries such as librosa and matplotlib. This way, community integration and development is eased.

Chapter 4

Implementation

In this chapter, we will explore the inner workings of MIRVisualScore through the lens of practical code examples and patterns drawn from this project's example scripts. We provide simplified code snippets to show architectural patterns, and workflows that demonstrate how the codebase is structured, how its components function, and how it is intended to be used in practice. We will progress through the essential topics: setting up the required dependencies, understanding and obtaining the required input files, examining how the layer-based visualization system works, and finally, exploring how to compose multiple data layers into aligned score visualizations. Through this approach, readers should gain a clear understanding of the project's core components and workflows, enabling them to adapt and extend the codebase for their own applications.

4.1 Dependencies and Setup

The dependencies and requirements and how to install them are laid out in the README.md of the codebase. Yet, we will go a bit more in depth in this section. MIRVisualScore as of now, runs using python 3.12.3, using other updated python environments might work, but this was the version used during the writing of this thesis. Using other python versions might cause conflicts with the rest of the codebase. For simplicity, we will run the codebase as if we were running it on a Linux machine, for running other operating systems, simply follow along as the README.md specifies.

We advise running this codebase with a virtual environment to avoid affecting other projects and repositories you might be working on your system.

Other than the python version the rest of the codebase requires the following dependencies:

- modusa==6.3.7
- matplotlib==3.10.9
- numpy==2.4.4
- librosa==0.11.0

- soundfile==0.13.1
- torch==2.11.0
- beat_this==1.1.0

To set this up, as the README.md explains, first make sure you have python 3.12.3 installed in your system. Then you can simply run the following commands in the terminal:

```
# Create virtual environment
python3.12 -m venv venv

# Activate the virtual enviroment
source venv/bin/activate

#Install requirements
pip install --upgrade pip
pip install -r requirements.txt
```

4.2 Required Files

For generating visualizations with audio and data aligned with the score we will, at least, three files: A audio file in .wav format with the performance recording, the warped score generated by ScoreWarp, the MAPS file, the .mei score and an .npz with BT data.

4.2.1 Getting the Warped Score

The warped score can be generated using the [ScoreWarp online demo](#). In it, one needs to upload the score in .mei format and the MAPS file in order to download the score in svg format.

The .mei file that contains the score in xml format. The easiest way to get a .mei score is to get the score in .musicxml format and then convert it to .mei, this can be done in multiple ways we will describe two: The first being through the [verovio online editor](#). The second, by using musescore's software that allows to then convert to .mei. In either way, it is important to get the original score in .musicxml format to be able to then convert it to .mei using the tools we described.

4.2.2 Getting the MAPS file

The MAPS file, describes the onsets of the performance and links them to each corresponding score elements of the .mei score through their @xml.id . Getting the maps file might be a bit more cumbersome, but we can describe two different ways of obtaining it:

The first is the one described in goeblLetsScoreWarpAgain2025a, using the MIDI-to-MIDI matching algorithm by nakamuraPERFORMANCEERRORDETECTION. However, this method

requires , along side with the .mei score, the performance in midi format. Using examples from the asap dataset peterAutomaticNoteLevelScoretoPerformance2023 will facilitate this process but if a midi performance is not possible to obtain then the maps file needs to be generated independently.

The MAPS file is at it's core a .json file, and for the purposes of the current MIRVisualScore version, it needs to provide the score's and performance onset data in the following manner:

```
{
  "obs_mean_onset": 0.853333333,
  "xml_id": ["xliwleq1"],
  "obs_num": 1
}
```

Where:

- “*obs_mean_onset*” is the time in seconds of a given onset.
- “*xml_id*” is the @xml.id of the corresponding onset note in the .mei score. In the case that the onset has more than one note, like a chord hit, this attribute can have more than one @xml.id.
- “*obs_num*” acts as a simple counter for each detected onset.

MIRVisualScore relies on MAPS in two complementary ways: directly for onset visualization, and indirectly through ScoreWarp, which requires MAPS to generate the warped score aligned with the timeline SVG element which constitutes the alignment reference for all overlaid data.

ScoreWarp performs best when MAPS contains dense onset annotations across the full performance. However, it can still align scores when annotations are sparse by interpolating between available onset and their @xml.id anchor points. In practice, this allows different precision levels (i.e., downbeats, phrase boundaries, or measure-level anchors), trading annotation effort for alignment precision. Even minimal anchors (first and last onset) may be usable for short performances or segments, but intermediate alignment becomes less reliable due to interpolation error.

For the examples that we provide in the making of our codebase we modified PP to obtain this data. PP’s “Piano Aligner” plugin jiangPIANOPRECISIONADVANCING2025 matches performance onsets to score elements defined by the .mei file. In PP these are displayed in a regular SV time instance layer. We made a simple modification where, this layer, instead of displaying each onset with it’s corresponding score position, “*measure + position_in_measure*”, it would display the @xml.id of each onset. With this we could then export the time instances as .txt files with “*obs_mean_onset+xml_id*”, then convert it into the MAPS format using “*TXT_to_Maps()*” function in @src/functions/warp_score.

4.2.3 Providing BT data

Since contemporary BT algorithms are ML based, they primarily function by generating a beat activation function, which is a continuous curve representing the probability of a beat occurring at

any given frame. Visualizing this underlying probability curve, rather than just the discrete beat assessments, allows us to see the algorithm’s internal confidence level, giving crucial insights on the inner workings of the model.

In `@src/functions/Beat_Layers`, the function “`Run_BeatThis()`” generates an `.npz` file that contains the required data for this to be later visualized. This particular function creates the `.npz` file generated through `BeatThis`. However, the `.npz` file format can serve as an intermediate for BT visual layers, as long as it is structured like it is described in “`Run_BeatThis()`”. The `.npz` should contain five key components from the BT model:

Field	Purpose
<code>beat_times</code>	Time coordinates (seconds) aligned with probability frames
<code>beat_probs</code>	Continuous beat activation function values
<code>downbeat_probs</code>	Continuous downbeat activation function values
<code>detected_beats</code>	Discrete beat positions derived from peak detection
<code>detected_downbeats</code>	Discrete downbeat positions from peak detection

The probabilistic fields capture the algorithm’s internal confidence across time, while the detected positions represent the final beat assessments. This separation enables visualization of both the continuous confidence landscape and discrete beat estimates.

4.2.4 Summarizing Required Files

We now know the purpose of each required file and how to get it. To summarize all the files that are directly involved in the example, we are constructing:

File	Description
<code>.wav</code>	File with the audio recording of the performance
<code>.mei</code>	XML Score of the musical piece
<code>.maps.json</code>	Contains data that links onsets with score elements of the <code>.mei</code>
<code>.svg</code>	The score in visual format generated by <code>ScoreWarp</code> (requires <code>.mei</code> and <code>.maps a priori</code>)
<code>.npz</code>	File with algorithmic BT data

4.3 Calling Modules, Methods and Functions

The codebase divides its functionalities into 4 main categories, where each, has its own dedicated python script. They are located inside `@src/functions` and are:

- **`visualization_system.py`**: Focused on establishing the visualization methods for rendering the visual elements, and for managing `svg` outputs.
- **`Spectrogram_layer.py`**: Responsible for rendering spectrogram visualizations of audio.
- **`Beat_Layers.py`**: Dedicated for classes and methods that are involved in the visualization of beat data.

- **warp_score.py**: For methods that play a rule in the score-to-audio alignment between the warp svg score and the other elements that can be displayed along side the score (the ones generated by the other categories).

In **visualization_system.py**, the “`class Layer(ABC)`” defines the abstract base class for all visual components. This class provides a common interface and structure for creating various visualizations, ensuring consistency and reusability across the codebase. The `Layer` class defines methods that layer methods should implement: `load_data()`, `draw()`, and `to_svg_group()`. Each of these three methods serves a distinct purpose in the visualization pipeline:

- `load_data(**kwargs)` — Loads and validates all necessary data for the layer. For instance: `class Spectrogram(Layer)` computes the mel-scale spectrogram from an audio file, `class BeatProbabilityLayer(BeatLayer)` loads BT data from the .npz file, and `Onsets_Layer` extracts onset times from a MAPS JSON file.
- `draw(ax, shared_data)` — Renders the visualization onto a matplotlib axis object. It accesses a `shared_data` dictionary that contains context information (such as audio duration, file paths, and SVG coordinate mappings) these are shared across all layers. This method returns a tuple of (lines, labels) which are collected for the legend.
- `to_svg_group(shared_data)` — Converts the rendered visualization into vector-based SVG markup. This is essential for creating composited visualizations that can be embedded within the score SVG. Not all layers support SVG export; those that do, return SVG group markup, while others return `None`.

To orchestrate multiple layers into a unified visualization, the `Visualizer` class acts as a central coordinator. It maintains a collection of `Layer` objects and manages the workflow:

- `add_layer(layer)` — Appends a layer to the visualization
- `load_all_layers(**kwargs)` — Sequentially loads data for all registered layers, and when an audio path is provided, extracts the audio duration and stores it in `shared_data`
- `draw()` — Renders all layers onto a single matplotlib figure, collects lines and labels for the legend, and enforces consistent time-axis limits across layers

The `shared_data` dictionary serves as a coordination mechanism between layers and export steps. It is initialized empty and populated during execution with values such as the audio duration, file paths, axis objects, and SVG conversion context. This lets different parts of the visualization pipeline to work together by using shared data, without needing to depend on each other directly. The composition pattern is best illustrated through a practical example. The following code creates a visualization combining different audio data layers:

```
#Initialize a visualizer instance.
```



```

example1=Visualizer()

# Add layers to the visulization.
example1.add_layer(Onsets_Layer(onset_color=(1, 1, 1),
                                line_width=0.5))
example1.add_layer(BeatAccurateLayer(beat_color=(1, 0, 0),
                                     downbeat_color=(0, 0, 1)))
example1.add_layer(BeatProbabilityLayer(color=(1, 0, 0)))

#Load data for layers.
example1.load_all_layers(audio_path  =audio_file_example1,
                        maps_file    =maps_file_example1,
                        beat_file    =beat_example1)

#Render the visualization:
fig, ax = example1.draw()

#Show visualization in Matplotlib window:
plt.show()

```

The above example illustrates the layer-based implementation philosophy. At its core, the code-base composes matplotlib visualizations and arranges them so they can be displayed alongside a musical score and audio representations. The project relies on the SVG format because Verovio (and subsequently ScoreWarp) render the score in SVG, besides, embedding external SVG score markup into matplotlib plots is not straight forward especially given the complexity of musical scores. Because SVG is vector-based, 2D graphics can be exported without losing resolution. However, some data types, (e.g. spectrograms), cant be presented in this way, thus we employed a way to render plots as PNG's as well. This trade-off motivated the focus on SVG format. This preserves score elements and most data layers with high fidelity, even tough certain elements might need to be rasterized.

Matplotlib generated layers have to afterwards be rendered into SVG. In the case that the layers do not require to be rasterized, by calling `Visualizer().TurnLayersIntoSVG()` we create an SVG render of the plots. In doing this, each layer is made into its own self-contained SVG `<g>` element in the xml structure, this is done automatically through the `to_svg_group()` method.

In the case of a layer that needs to be rasterized, i.e., a spectrogram, the plot is rendered into a PNG using `Visualizer().TurnLayersIntoPNG()`, so that then, it can be embedded into a SVG using `Visualizer().Align_Score_and_PNG()`. This method takes care of embedding the PNG into the SVG and aligning it with the score.

Afterwards a visualization in SVG can combine the rendered elements into a single combined SVG, for this we call `Visualizer().combine_AlignedScore_with_Layers()` which takes care of merging the score and all imported layers into a single SVG file. In it, the layers (including the rasterized ones) are embedded into the same xml class `Layers` keeping the XML structure previously generated through `Visualizer().TurnLayersIntoSVG()`.

Finally, a complete multi-visualization composite can be rendered through `Visualizer().create_final_SVG()`. This method combines different provided SVG files into a single one. It keeps the same XML structure philosophy, this time, one level above, i.e., each previously rendered SVG keeps the same structure and are treated as independent children of a new added root. In this final step, visualization can be offset vertically to allow for side-by-side comparisons.

In a sense, the codebase can be characterized by this layer-based philosophy, where layers are visual elements that can be combined and arranged. The project does this at two different levels, the first is at its core a matplotlib plot generator, and the second treats SVG elements as upper-level visual layers where they can be combined and arranged as individual elements.

4.4 Creating the Audio Visualizations

4.5 Score and Data Alignment

In this section we'll go in depth on how the score and data visualization alignment functions at it's core.

4.6 Combining Visualizations

4.7 Conclusion

Chapter 5

Conclusions and Future Work

5.1 Achievements

5.2 Future Work

References

- [1] *An Introduction to MEI*. URL: <https://music-encoding.org/about/>.
- [2] Yucong Jiang and Werner Goebel. “PIANO PRECISION: ADVANCING MUSIC PERFORMANCE ANALYSIS BY INTEGRATING USER-CORRECTABLE AUDIO-TO-SCORE ALIGNMENT”. In: (2025).
- [3] Jongmin Jung et al. “Unified Cross-modal Translation of Score Images, Symbolic Music, and Performance Audio”. In: *IEEE Transactions on Audio, Speech and Language Processing* 34 (2026), pp. 1876–1891. ISSN: 2998-4173. DOI: [10.1109/TASLPRO.2025.3648794](https://doi.org/10.1109/TASLPRO.2025.3648794). arXiv: [2505.12863](https://arxiv.org/abs/2505.12863) [cs.SD]. URL: <http://arxiv.org/abs/2505.12863> (visited on 06/02/2026).
- [4] Zulfidin Khodzhaev. “A Practical Guide to Spectrogram Analysis for Audio Signal Processing”. In: ().
- [5] Meinard Müller. *Fundamentals of Music Processing: Audio, Analysis, Algorithms, Applications*. Cham: Springer International Publishing, 2015. ISBN: 978-3-319-21944-8 978-3-319-21945-5. DOI: [10.1007/978-3-319-21945-5](https://doi.org/10.1007/978-3-319-21945-5). URL: <https://link.springer.com/10.1007/978-3-319-21945-5> (visited on 06/02/2026).
- [6] Meinard Müller. “Music Representations”. In: *Fundamentals of Music Processing*. Cham: Springer International Publishing, 2015, pp. 1–37. ISBN: 978-3-319-21944-8 978-3-319-21945-5. DOI: [10.1007/978-3-319-21945-5_1](https://doi.org/10.1007/978-3-319-21945-5_1). URL: https://link.springer.com/10.1007/978-3-319-21945-5_1 (visited on 06/02/2026).
- [7] Silvan David Peter et al. “Automatic Note-Level Score-to-Performance Alignments in the ASAP Dataset”. In: *Transactions of the International Society for Music Information Retrieval* 6.1 (June 26, 2023), pp. 27–42. ISSN: 2514-3298. DOI: [10.5334/tismir.149](https://doi.org/10.5334/tismir.149). URL: <http://transactions.ismir.net/articles/10.5334/tismir.149/> (visited on 06/07/2026).
- [8] Rafael Valle. “Visual Display and Retrieval of Music Information”. In: ().
- [9] K. Yamamoto. “Human-in-the-Loop Adaptation for Interactive Musical Beat Tracking”. In: *Proceedings of the 22nd International Society for Music Information Retrieval Conference (ISMIR)*. 2021.

Appendix A

Lorem Ipsum

After the conclusions and bibliographical references, the text used to complete the dissertation is presented in this numbered annex.

A.1 What is *Lorem Ipsum*?

Lorem Ipsum is simply dummy text of the printing and typesetting industry. Lorem Ipsum has been the industry's standard dummy text ever since the 1500s, when an unknown printer took a galley of type and scrambled it to make a type specimen book. It has survived not only five centuries, but also the leap into electronic typesetting, remaining essentially unchanged. It was popularised in the 1960s with the release of Letraset sheets containing Lorem Ipsum passages, and more recently with desktop publishing software like Aldus PageMaker including versions of Lorem Ipsum [kn:Lip08].

A.2 Where does *Lorem* come from?

Contrary to popular belief, Lorem Ipsum is not simply random text. It has roots in a piece of classical Latin literature from 45 BC, making it over 2000 years old. Richard McClintock, a Latin professor at Hampden-Sydney College in Virginia, looked up one of the more obscure Latin words, consectetur, from a Lorem Ipsum passage, and going through the cites of the word in classical literature, discovered the undoubtable source. Lorem Ipsum comes from sections 1.10.32 and 1.10.33 of “de Finibus Bonorum et Malorum” (The Extremes of Good and Evil) by Cicero, written in 45 BC. This book is a treatise on the theory of ethics, very popular during the Renaissance. The first line of Lorem Ipsum, “Lorem ipsum dolor sit amet. . .”, comes from a line in section 1.10.32.

The standard chunk of Lorem Ipsum used since the 1500s is reproduced below for those interested. Sections 1.10.32 and 1.10.33 from “de Finibus Bonorum et Malorum” by Cicero are also reproduced in their exact original form, accompanied by English versions from the 1914 translation by H. Rackham.

A.3 Why is *Lorem* used?

It is a long established fact that a reader will be distracted by the readable content of a page when looking at its layout. The point of using Lorem Ipsum is that it has a more-or-less normal distribution of letters, as opposed to using “Content here, content here”, making it look like readable English. Many desktop publishing packages and web page editors now use Lorem Ipsum as their default model text, and a search for “lorem ipsum” will uncover many web sites still in their infancy. Various versions have evolved over the years, sometimes by accident, sometimes on purpose (injected humour and the like).

A.4 Where can you find examples?

There are many variations of passages of Lorem Ipsum available, but the majority have suffered alteration in some form, by injected humour, or randomised words which don't look even slightly believable. If you are going to use a passage of Lorem Ipsum, you need to be sure there isn't anything embarrassing hidden in the middle of text. All the Lorem Ipsum generators on the Internet tend to repeat predefined chunks as necessary, making this the first true generator on the Internet. It uses a dictionary of over 200 Latin words, combined with a handful of model sentence structures, to generate Lorem Ipsum which looks reasonable. The generated Lorem Ipsum is therefore always free from repetition, injected humour, or non-characteristic words etc.